

Docker

Docker packages an application together with everything it needs to run, so the same package behaves the same way on a laptop, in a pipeline and in production.

1. What Docker is

Docker container is a lightweight, isolated environment that runs an application along with its dependencies. Docker is the platform that builds those containers and runs them.

The problem Docker removes is the oldest one in deployment. An application works on a developer's laptop and fails on the server, because the two machines have a different Java version, a different library, a different locale or a different path. Docker takes the disagreement away by shipping the environment with the application instead of assuming it is already there.

Three words carry the whole idea:

Image → the package: the application plus its dependencies
Container → that package, running as an isolated process
Registry → where images are stored and shared

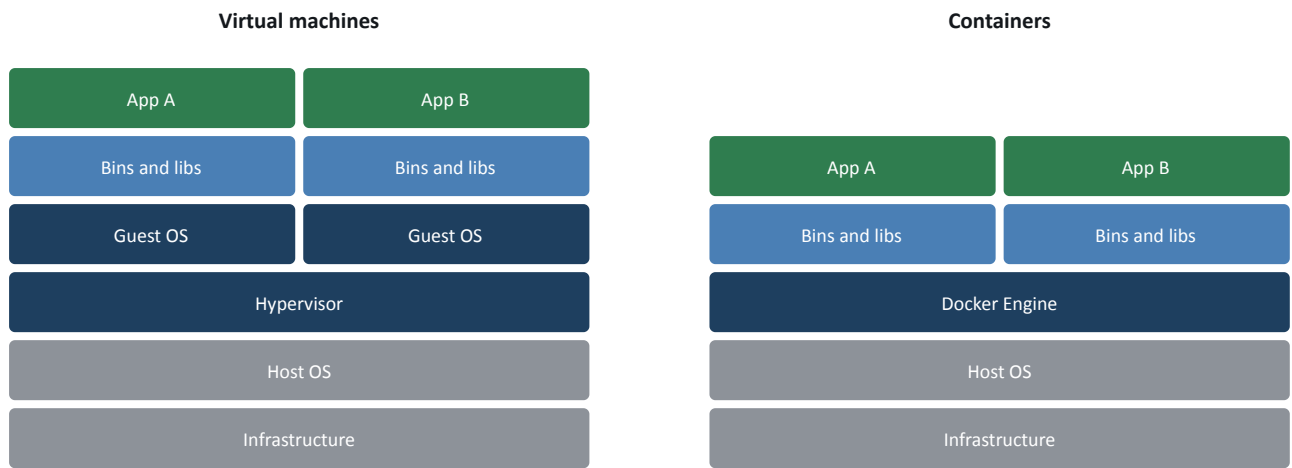
A build produces an image. The image is pushed to a registry. Any machine with Docker pulls that image and runs it as a container. Nothing is installed on the machine except Docker itself.

A container is a process, not a machine. The isolated environment is real, but it is built by restricting a normal Linux process: namespaces decide what the process can see, and cgroups decide what it can use. That one sentence explains the size, the start time and the limits of everything that follows.

2. Docker against a virtual machine

Container virtualizes the application environment, while a Virtual Machine virtualizes an entire operating system.

A virtual machine needs a hypervisor and a second operating system to do it. A container needs neither, because the kernel is already there.



Each virtual machine carries its own operating system. Containers share the host's, which is the whole of the difference in size and start time.

Docker Container	Virtual Machine
Shares the host OS kernel	Has its own OS
Lightweight	Heavier
Starts in seconds or less	Usually takes longer to boot
Uses fewer resources	Uses more resources
Smaller size	Larger size
Good for microservices	Good for full OS isolation

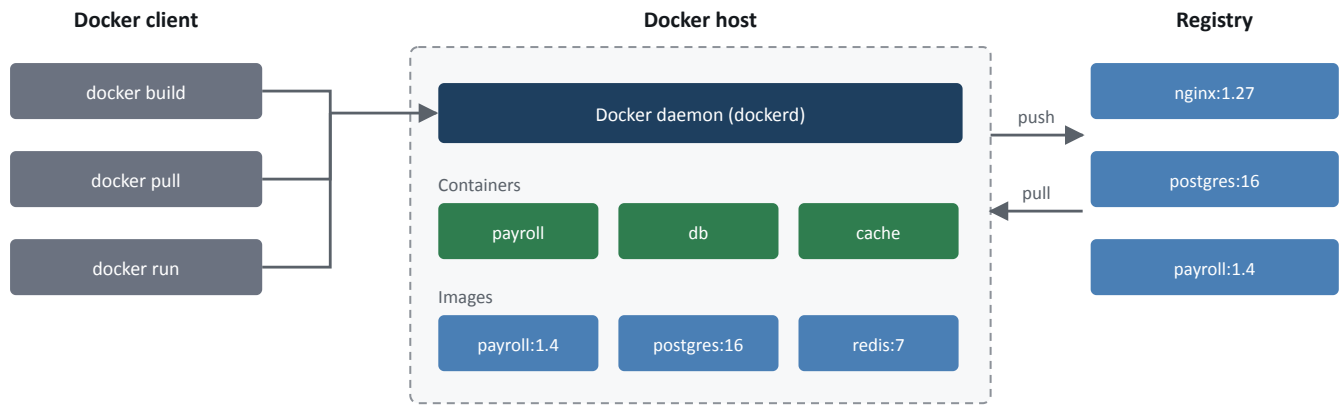
A container is not a small virtual machine. There is no guest operating system inside it and no boot sequence to wait for.

That also explains the limit: a Linux container needs a Linux kernel. On macOS and Windows, Docker runs a small Linux virtual machine and puts the containers inside it, which is why the first start on a laptop is slow and every start after it is not.

Neither one replaces the other. A virtual machine gives stronger isolation, because a container escape reaches the shared kernel. Cloud providers run our containers inside their virtual machines for exactly this reason.

3. Docker architecture

Docker is a client-server system. The `docker` command we type is a thin client, and a daemon on the host does the work.



Every command goes to the daemon. The daemon holds the images and the containers, and it is the daemon, never the client, that reaches the registry.

Piece	What it is
Docker client	The <code>docker</code> command. Sends requests, shows the answers
Docker daemon	<code>dockerd</code> . Builds images, runs containers, manages volumes and networks
Docker host	The machine the daemon runs on
Registry	The server that stores images. Docker Hub is the default

4. The Docker Engine

Docker Engine is the software that creates, runs, and manages Docker containers. When we say "we installed Docker", the engine is what got installed.

It mainly consists of:

- **Docker Daemon** → Creates and manages containers, images, networks, and volumes.
- **Docker CLI** → Allows us to interact with Docker using commands like `docker run` .
- **Docker API** → Allows applications to communicate with the Docker daemon.

The daemon does not create containers by itself. It delegates downward:

```

docker (CLI)
├── dockerd      images, volumes, networks, the API
│   └── containerd  container lifecycle: start, stop, pull
│       └── runc      talks to the kernel, creates the process

```

`runc` is the piece that actually asks the Linux kernel for the namespaces and the cgroups. It runs, creates the container, and exits.

This layering is why Kubernetes can drop Docker and keep running containers. Kubernetes talks to `containerd` directly and skips `dockerd`. The images we build are unaffected, because the image format is a standard that the whole stack agrees on.

Docker Desktop is not the engine. It is the engine plus the small Linux virtual machine it needs on macOS and Windows, plus a user interface.

5. The client and the daemon

Docker Client sends commands; Docker Daemon executes and manages them.

Docker Client	Docker Daemon
Used to interact with Docker	Does the actual Docker work
Usually the <code>docker</code> CLI	Runs as a background service
Sends requests	Receives and processes requests
Example: <code>docker run nginx</code>	Creates and runs the container

They are two separate programs, and `docker version` reports on both:

```
docker version
```

Output:

```
Client:
 Version:   27.3.1
 OS/Arch:   darwin/arm64

Server: Docker Desktop
 Engine:
  Version:   27.3.1
  OS/Arch:   linux/arm64
 containerd:
  Version:   1.7.21
 runc:
  Version:   1.1.13
```

Two separate operating systems, as well. The client is a macOS program; the server is Linux, inside the virtual machine from section 2.

They talk over a socket. On Linux and macOS it is the Unix socket `/var/run/docker.sock`. Every command is an HTTP call on it, and the API can be called without the client at all:

```
curl --unix-socket /var/run/docker.sock http://localhost/containers/json
```

Three consequences are worth carrying into an interview.

The build context is sent to the daemon. `docker build .` uploads the directory first, which is why a missing `.dockerignore` makes builds slow, and why a Dockerfile cannot copy a file from outside the context.

The daemon can be remote. `DOCKER_HOST` or `-H` points the client at another machine, and the build then happens there.

```
docker -H ssh://deploy@build-server ps
```

Access to the socket is root on the host. A container given `-v /var/run/docker.sock:/var/run/docker.sock` can start another container that mounts the whole host filesystem. Mount it only when a tool genuinely needs it.

6. Image against container

Docker Image is the blueprint; Docker Container is the running instance of that blueprint.

These two words get used for each other, and the distinction is the first thing an interview asks.

Docker Image	Docker Container
Read-only template	Running/created instance
Contains application + dependencies	Runs the application
Used to create containers	Created from an image
Doesn't run by itself	Can be started, stopped, and deleted
Example: <code>nginx:latest</code>	Example: a running Nginx container

One image, many containers, exactly as one class gives many objects.

```
docker run --name first -d nginx:1.27
docker run --name second -d nginx:1.27
```

Two containers, one image. Neither can change the image. Each gets its own thin writable layer on top.

Each word has its own commands, and mixing them up is the usual first mistake: `docker build` makes an image and `docker run` makes a container, `docker image ls` lists images and `docker ps` lists containers, `docker rmi` deletes an image and `docker rm` deletes a container.

Writing inside a container is not permanent. Files written to the writable layer disappear when the container is removed. Section 12 covers what to do instead.

7. Registries and Docker Hub

Docker Registry is a storage system where Docker images are stored and shared. It is to images what a Maven repository is to jars.

```
docker.io/library/nginx:1.27
└─ registry ─┬─ repo ─┬─ tag┘
```

Part	Meaning	Default
Registry	The host that stores the image	docker.io
Repository	A name holding all versions of one image	—
Tag	The version inside that repository	latest

So `nginx` alone means `docker.io/library/nginx:latest`. Everything omitted has a default, which is convenient and also the source of a common surprise.

Docker Hub is a cloud-based public Docker Registry used to store, share, and download Docker images. It is also the default one. It holds the official images, which are the ones with no user prefix: `nginx`, `postgres`, `eclipse-temurin`. An image under a user name, such as `bitnami/postgresql`, is published by that account and carries no such review.

```
docker pull postgres:16           # from Docker Hub
docker login registry.company.com  # sign in to a private registry
docker tag payroll:1.4 registry.company.com/team/payroll:1.4
docker push registry.company.com/team/payroll:1.4
```

`docker tag` does not copy anything. It adds a second name to the same image, and the name is what decides where a push goes.

Most companies run a private registry instead: Amazon ECR, Google Artifact Registry, Azure Container Registry, or Harbor and Nexus when it is self hosted. The reasons are access control, images that must not be public, pull speed inside the network, and Docker Hub's rate limit on anonymous pulls, which a busy pipeline reaches quickly.

Never rely on `:latest`. It is a tag like any other and it moves. A build that worked yesterday can pull a different image today. Pin a real version, and pin the base image in the Dockerfile as well.

8. The Dockerfile

A Dockerfile is the recipe for an image. Each instruction produces a layer.

```
FROM eclipse-temurin:21-jre-alpine

WORKDIR /app

COPY target/payroll.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Instruction	What it does
FROM	The image to start from
WORKDIR	The directory the next instructions run in
COPY	Copies files from the build context into the image
RUN	Runs a command while building
EXPOSE	Documents the port. It publishes nothing by itself
ENV	Sets an environment variable
ENTRYPOINT	The command the container runs
CMD	Default arguments, or the command when there is no entrypoint

ENTRYPOINT and **CMD** are not alternatives. **ENTRYPOINT** is the program and **CMD** is its default arguments, so `docker run image --debug` replaces the arguments and keeps the program.

Use the exec form, not the shell form.

```
ENTRYPOINT ["java", "-jar", "app.jar"]    # the JVM is process 1
ENTRYPOINT java -jar app.jar              # a shell is process 1
```

The shell form wraps the command in `/bin/sh -c`, so the JVM is not process 1 and never receives the stop signal. The container then takes the full ten seconds to be killed on every deploy.

A multi-stage build keeps the toolchain out of the result.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /src/target/ payroll.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Only the final stage becomes the image. Maven, the source and the local repository are left behind, which takes the result from around 700 MB to around 200 MB.

9. Building the image

```
docker build -t payroll:1.4 .
```

The `.` is the build context: the directory sent to the daemon before anything runs. A `.dockerignore` file keeps `target/`, `.git/` and `node_modules/` out of it.

```
target/
.git/
node_modules/
*.log
```

`-t` names the result. Without it the image has an id and nothing else, and it becomes hard to find within a day.

```
docker image ls
```

Output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
payroll	1.4	7c9e2a1f5b3d	12 seconds ago	214MB
postgres	16	d84b1f2a7c60	2 weeks ago	438MB

10. Layers and the build cache

Every instruction adds a layer, and layers are cached. On a rebuild, Docker reuses each layer until it reaches one whose inputs changed, then rebuilds that one and everything after it.

That ordering is most of what makes a rebuild fast or slow:

```
# slow: any source change re-downloads every dependency
COPY . .
RUN mvn package

# fast: dependencies are cached until pom.xml changes
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package
```

Put what changes least at the top. Dependencies change monthly and source changes hourly, so the dependency layer should come first and be reused.

Layers are also shared between images. Ten images built **FROM** the same base store that base once.

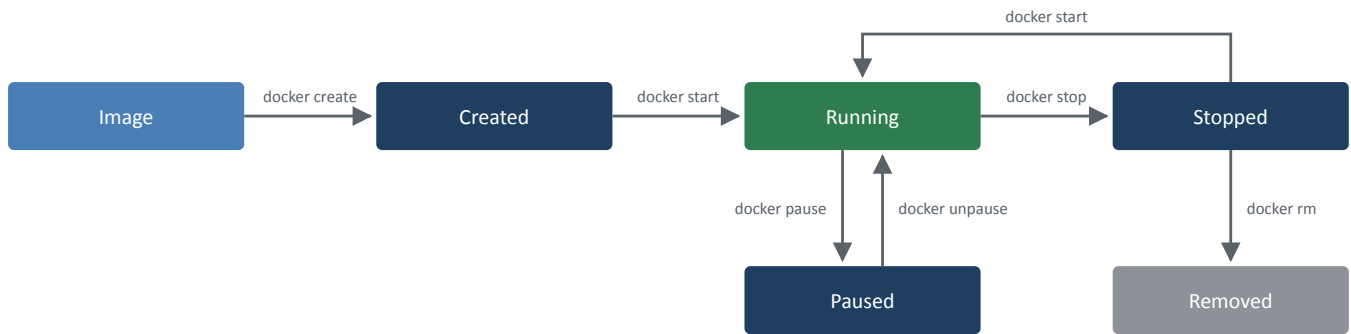
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
payroll	1.4	7c9e2a1f5b3d	2 minutes ago	214MB
payroll	1.3	2b8d41c7e9a0	3 days ago	214MB

Those two are not 428 MB on disk. They share every layer except the one holding the jar.

A deleted file still occupies space. Removing something in a later layer hides it, and the earlier layer still carries it, which is why a secret copied in and deleted later is still in the image.

11. The container lifecycle

The Docker Container Lifecycle describes the different states a container goes through from creation to removal.



A stopped container still exists. It keeps its writable layer, its name and its logs until it is removed.

1. Created. Container is created but not running.

```
docker create nginx
```

2. Running. Container is actively running.

```
docker start <container>
```

Or create and start it together:

```
docker run nginx
```

3. Paused. Container's processes are temporarily paused.

```
docker pause <container>    # freeze
docker unpause <container>   # resume
```

4. Stopped. Container exists, but its main process is not running. It can be started again.

```
docker stop <container>
docker start <container>
```

5. Removed. Container is permanently deleted.

```
docker rm <container>
```

In practice `docker run` is what gets typed, with the flags that shape the container:

```
docker run -d --name payroll -p 8080:8080 payroll:1.4
```

Flag	What it does
<code>-d</code>	Runs in the background
<code>--name</code>	Names the container, instead of a random one
<code>-p 8080:8080</code>	Maps host port to container port
<code>-e KEY=value</code>	Sets an environment variable
<code>-v data:/var/lib/data</code>	Mounts a volume
<code>--rm</code>	Deletes the container when it stops
<code>--restart unless-stopped</code>	Starts it again after a crash or a reboot

```
docker ps -a
```

Output:

CONTAINER ID	IMAGE	COMMAND	STATUS	NAMES
a3f1c82b9e04	payroll:1.4	"java -jar app.jar"	Up 2 minutes	payroll
7d20e4b6c115	payroll:1.3	"java -jar app.jar"	Exited (137) 4 hours ago	
payroll-old				

`docker ps` shows running containers; `-a` adds the stopped ones. The exit code in brackets is the answer to most questions: `0` is a clean exit, `1` is the application failing, and `137` means the process was killed, which on a server usually means it ran out of memory.

A container lives exactly as long as its main process. When that process exits, the container stops. A container that will not stay running is almost always a process that finished or crashed, and `docker logs` says which.

`docker stop` is polite, `docker kill` is not. `stop` sends `SIGTERM`, waits ten seconds, then sends `SIGKILL`. Those ten seconds are the application's chance to finish requests and close connections, so the process must be process 1 to receive the signal at all. `docker kill` sends `SIGKILL` at once and skips the shutdown.

12. Keeping data

A container's writable layer is deleted with the container. Anything that must survive goes outside it.

Kind	Written as	Use it for
Volume	<code>-v pgdata:/var/lib/postgresql/data</code>	Data Docker manages. The normal choice
Bind mount	<code>-v \$(pwd)/src:/app/src</code>	A host directory, for development
tmpfs	<code>--tmpfs /tmp</code>	Memory only, never written to disk

```
docker volume create pgdata

docker run -d --name db \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

The container can now be deleted and recreated, and the database is still there.

Prefer a volume to a bind mount for real data. A bind mount depends on a path on one machine, and its file ownership differs between macOS and Linux.

13. Networking and ports

`-p 8080:9090` reads host first, container second.

```
docker run -p 8080:8080 payroll:1.4    # reachable at localhost:8080
```

`EXPOSE` in a Dockerfile documents the port and opens nothing. Only `-p` does.

Containers on the same user-defined network reach each other **by container name**, which is what removes hardcoded addresses:

```
docker network create payroll-net

docker run -d --name db      --network payroll-net postgres:16
docker run -d --name payroll --network payroll-net \
  -e DB_URL=jdbc:postgresql://db:5432/payroll payroll:1.4
```

`db` resolves through Docker's own DNS. The default bridge network does not do this, which is why a user-defined network is worth creating.

`localhost` inside a container is the container. An application trying to reach the host database on `localhost` finds nothing; the name is `host.docker.internal` on macOS and Windows.

14. Docker Compose

One command for several containers, described in a file instead of remembered.

```

services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 5

```

The application goes in the same file, below the database:

```

payroll:
  build: .
  ports:
    - "8080:8080"
  environment:
    DB_URL: jdbc:postgresql://db:5432/payroll
  depends_on:
    db:
      condition: service_healthy

volumes:
  pgdata:

```

```

docker compose up -d      # start everything
docker compose logs -f    # follow the logs
docker compose down       # stop and remove
docker compose down -v    # and delete the volumes

```

Compose creates a network for the project, so `db` resolves with no extra configuration.

`depends_on` alone only waits for the container to start, not for the database to accept connections. The `condition: service_healthy` form, with a healthcheck, is what actually waits.

15. The commands worth knowing

Command	What it does
<code>docker build -t name:tag .</code>	Builds an image from the Dockerfile here
<code>docker run -d -p 8080:8080 name:tag</code>	Starts a container in the background
<code>docker ps</code> / <code>docker ps -a</code>	Running containers, or all of them

Command	What it does
<code>docker logs -f name</code>	Follows a container's output
<code>docker exec -it name sh</code>	Opens a shell inside a running container
<code>docker stop name</code> / <code>docker rm name</code>	Stops it, then deletes it
<code>docker restart name</code>	Stops it and starts it again
<code>docker image ls</code> / <code>docker rmi id</code>	Lists images, deletes one
<code>docker pull name:tag</code> / <code>docker push name:tag</code>	Moves an image to or from a registry
<code>docker inspect name</code>	Everything the daemon knows about it, as JSON
<code>docker stats</code>	Live CPU and memory use, per container
<code>docker cp name:/app/log.txt .</code>	Copies a file out of a container, or into one
<code>docker system df</code>	Shows what is using disk
<code>docker system prune -a</code>	Deletes everything unused

`docker exec -it name sh` is the one that answers most questions. It puts a shell inside the running container, where the filesystem, the environment variables and the network are exactly what the application sees.

```
docker exec -it payroll sh
/app # ls
app.jar
/app # env | grep DB_URL
DB_URL=jdbc:postgresql://db:5432/payroll
```

The pieces together

Piece	What it answers
Client and daemon	Who does the work, and where
Engine	What builds and runs containers
Image	What is packaged
Container	What is running
Registry	Where images are stored and shared
Dockerfile	How the image is built
Layer cache	Why a rebuild is fast or slow
Lifecycle	How a container starts, stops and is deleted

Piece	What it answers
Volume	What survives the container
<code>-p</code>	How the outside reaches in
User-defined network	How containers reach each other
Compose	How several start together

For example, imagine the payroll service. A multi-stage Dockerfile builds the jar with Maven and copies only the jar into a JRE image, so the result is around 200 MB. `pom.xml` is copied before the source, so a code change rebuilds one layer rather than re-downloading every dependency. The client sends the build to the daemon, the daemon pushes the image to the company registry, and the server pulls it from there. Compose starts it beside PostgreSQL on a project network, where the application reaches the database as `db`. The database writes to a named volume, so `docker compose down` and back up keeps the data.

The most useful thing at this point is not to memorize the flags. Take one application, write a Dockerfile for it, and then reorder the `COPY` lines and watch the rebuild time change.